

# Problema2

January 5, 2025

## 0.1 Problema 2:

### 0.1.1 Este exercício é dirigido à prova de correção do algoritmo estendido de Euclides apresentado no trabalho TP3

- a) Construa a asserção lógica que representa a pós-condição do algoritmo. Note que a definição da função

gcd

é

$$\text{gcd}(a, b) \equiv \min\{r > 0 \mid \exists s, t. r = a * s + b * t\}$$

.

```
[ ]: from z3 import *

def verifica_pos_condicao(a_val, b_val, r_val, s_val, t_val):
    # Criar solver
    solver = Solver()

    # Declarar variáveis
    a = Int('a')
    b = Int('b')
    r = Int('r')
    s = Int('s')
    t = Int('t')

    # Adicionar valores concretos
    solver.add(a == a_val)
    solver.add(b == b_val)
    solver.add(r == r_val)
    solver.add(s == s_val)
    solver.add(t == t_val)

    # Pós-condição
    pos_cond = And(
        r > 0, # r é positivo
        r == a * s + b * t, # r é combinação linear de a e b
        ForAll([Int('x'), Int('y')], # Para todo x,y
            Implies(
```

```

        And(
            Int('x') == a * Int('y') + b * Int('y'),
            Int('x') > 0
        ),
        Int('x') >= r      # r é o menor valor positivo
    )
)

)

solver.add(pos_cond)

# Verificar
if solver.check() == sat:
    modelo = solver.model()
    return True, f"Pós-condição satisfeita: {r_val} = {a_val}*{s_val} +_
↳{b_val}*{t_val}"
else:
    return False, "Pós-condição não satisfeita"

def teste():
    casos_teste = [
        (48, 36, 12, 1, -1),      # Caso válido
        (17, 5, 1, -2, 7),       # Caso válido
        (48, 36, 12, -1, 2)      # Caso inválido
    ]

    for a, b, r, s, t in casos_teste:
        print(f"\nTestando a={a}, b={b}, r={r}, s={s}, t={t}")
        sucesso, mensagem = verifica_pos_condicao(a, b, r, s, t)
        print(f"Resultado: {'Sucesso' if sucesso else 'Falha'}")
        print(f"Mensagem: {mensagem}")

if __name__ == "__main__":
    print("Testando Pós-condição com Z3:")
    teste()

```

Testando Pós-condição com Z3:

Testando a=48, b=36, r=12, s=1, t=-1

Resultado: Sucesso

Mensagem: Pós-condição satisfeita: 12 = 48\*1 + 36\*-1

Testando a=17, b=5, r=1, s=-2, t=7

Resultado: Sucesso

Mensagem: Pós-condição satisfeita: 1 = 17\*-2 + 5\*7

Testando a=48, b=36, r=12, s=-1, t=2

Resultado: Falha

Mensagem: Pós-condição não satisfeita

Para a realização deste exercício utilizamos a biblioteca `z3` como auxílio. Definimos a função `verifica_pos_condicao()` que recebe 5 valores: `'a_val'`, `'b_val'`, `'r_val'`, `'s_val'` e `'t_val'`. Esta verifica se os valores satisfazem uma pós condição específica.

É criado o solver através da variável `solver = Solver()` que é responsável por resolver o problema dado.

São definidas as variáveis do tipo “Int”. E fazemos atribuições dos valores fornecidos às variáveis, que permite que o solver use estes mesmos valores no processo de verificação.

De seguida definimos a pós-condição que é composta por três condições:

1.  $r > 0$ :

A condição exige que o valor de `r` seja sempre positivo, caso contrário a mesma não vai funcionar.

2. Combinação linear das variáveis `a` e `b`:

## 0.2 $r = as + bt$

Esta condição expressa a variável `r` como uma combinação linear das variáveis `a` e `b` com os coeficientes `s` e `t`.

3. Condição sobre `x` e `y`:

Para todos os inteiros `x` e `y`, se a condição:

## 0.3 $x = ay + bz > 0$

for satisfeita então garantimos que:

## 0.4 $x \geq r$

Esta condição exige que qualquer valor `x` que resulte da combinação linear de `a` e `b` é maior ou igual a `r`.

4. Verificação da pós-condição:

Depois de adicionar a pós-condição ao solver, o código vai verificar se o problema é `sat` ou `unsat` (satisfazível ou insatisfazível). Caso seja satisfazível o programa retorna o valor booleano `True` com uma mensagem a dizer que a pós-condição foi satisfeita. Caso contrário é devolvido o valor booleano `False` com uma mensagem a dizer que a pós-condição não foi satisfeita.

Para finalizar a alínea a) fizemos um programa simples que testa diferentes conjuntos de valores.

- b) Usando a metodologia do comando `havoc` para o ciclo, escreva o programa na linguagem dos comandos anotados (LPA). Codifique a pós-condição do algoritmo com um comando `assert`.

```
[15]: from z3 import *

def lpa(a_val, b_val):
    solver = Solver()

    # Criar variáveis simbólicas para o estado inicial
```

```

a = Int('a')
b = Int('b')
r = Int('r')
r_linha = Int('r_linha')
s = Int('s')
s_linha = Int('s_linha')
t = Int('t')
t_linha = Int('t_linha')
q = Int('q')

# Estado inicial
solver.add(a == a_val)
solver.add(b == b_val)
solver.add(r == a_val)
solver.add(r_linha == b_val)
solver.add(s == 1)
solver.add(s_linha == 0)
solver.add(t == 0)
solver.add(t_linha == 1)

# Invariante inicial
solver.add(r == a * s + b * t)
solver.add(r_linha == a * s_linha + b * t_linha)

estados = []
r_atual, r_linha_atual = a_val, b_val
s_atual, s_linha_atual = 1, 0
t_atual, t_linha_atual = 0, 1

while r_linha_atual != 0:
    q = r_atual // r_linha_atual

    # Criar novas variáveis para o próximo estado
    r_prox = Int(f'r_{len(estados)}')
    r_linha_prox = Int(f'r_linha_{len(estados)}')
    s_prox = Int(f's_{len(estados)}')
    s_linha_prox = Int(f's_linha_{len(estados)}')
    t_prox = Int(f't_{len(estados)}')
    t_linha_prox = Int(f't_linha_{len(estados)}')

    # Adicionar restrições de transição
    solver.add(r_prox == r_linha_atual)
    solver.add(r_linha_prox == r_atual - q * r_linha_atual)
    solver.add(s_prox == s_linha_atual)
    solver.add(s_linha_prox == s_atual - q * s_linha_atual)
    solver.add(t_prox == t_linha_atual)
    solver.add(t_linha_prox == t_atual - q * t_linha_atual)

```

```

    # Atualizar valores atuais
    r_atual, r_linha_atual = r_linha_atual, r_atual - q * r_linha_atual
    s_atual, s_linha_atual = s_linha_atual, s_atual - q * s_linha_atual
    t_atual, t_linha_atual = t_linha_atual, t_atual - q * t_linha_atual

    estados.append((r_atual, r_linha_atual, s_atual, s_linha_atual, t_atual,
→t_linha_atual, q))

    # Verificar se o modelo é satisfatível
    if solver.check() == sat:
        return True, estados
    else:
        return False, "Não foi possível encontrar uma solução válida"

def testar_lpa():
    casos_teste = [
        (48, 36),
        (17, 5),
        (21, 14)
    ]

    for a, b in casos_teste:
        print(f"\nExecutando programa LPA para a={a}, b={b}")
        sucesso, resultado = lpa(a, b)

        if sucesso:
            print("Estados gerados:")
            for i, estado in enumerate(resultado):
                r, r_linha, s, s_linha, t, t_linha, q = estado
                print(f"Estado {i}: r={r}, r'={r_linha}, s={s}, s'={s_linha},
→t={t}, t'={t_linha}, q={q}")
            print(f"MDC encontrado: {resultado[-1][0]}")
        else:
            print(f"Erro: {resultado}")

if __name__ == "__main__":
    print("Teste do programa LPA:")
    testar_lpa()

```

Teste do programa LPA:

Executando programa LPA para a=48, b=36

Estados gerados:

Estado 0: r=36, r'=12, s=0, s'=1, t=1, t'=-1, q=1

Estado 1: r=12, r'=0, s=1, s'=-3, t=-1, t'=4, q=3

MDC encontrado: 12

Executando programa LPA para a=17, b=5

Estados gerados:

Estado 0: r=5, r'=2, s=0, s'=1, t=1, t'=-3, q=3

Estado 1: r=2, r'=1, s=1, s'=-2, t=-3, t'=7, q=2

Estado 2: r=1, r'=0, s=-2, s'=5, t=7, t'=-17, q=2

MDC encontrado: 1

Executando programa LPA para a=21, b=14

Estados gerados:

Estado 0: r=14, r'=7, s=0, s'=1, t=1, t'=-1, q=1

Estado 1: r=7, r'=0, s=1, s'=-2, t=-1, t'=3, q=2

MDC encontrado: 7

Para esta alínea implementamos o Algoritmo de Euclides estendido utilizando o solver da biblioteca `z3`. Este programa calcula o máximo divisor comum ou MDC de dois números inteiros e em simultâneo determina os coeficientes da combinação linear que resulta no MDC.

Definimos primeiramente a função do programa, `lpa()` que recebe os valores `a_val` e `b_val`. São definidas as seguintes variáveis: `a`, `b`, `r`, `s`, `t`, `q`. Definimos também o solver que é caracterizado com estado inicial, onde:

- `r == a_val`;
- `r_linha == b_val`;
- `s`, `s_linha`, `t` e `t_linha` recebem valores iniciais que garantem as combinações lineares.

São adicionadas ao solver as restrições que garantem a validade das combinações lineares, ou seja, são adicionadas os seguintes invariantes:

- `r = a * s + b * t`
- `r_linha = a * s_linha + b * t_linha`

Para a iteração dos estados utilizamos um *while loop* que é um ciclo que permite calcular os próximos estados até que o valor de `r_linha`(que é o resto) seja zero. É calculado o quociente `q`, define-se o próximo estado com novas variáveis e por fim é atualizado os valores atuais com base nas transições.

Para o proximo passo verificamos se o modelo é satisfazível, assim como na alínea anterior se for `sat` é satisfazível e retorna os estados gerados, caso contrário é `unsat` insatisfazível e devolve uma mensagem a dizer que não é possível encontrar uma solução válida.

Para finalizar esta alínea, assim como na alínea o grupo fez uma programa que testa com um conjunto de valores a viabilidade do programa realizado.

- c) Construa codificações do programa LPA através de transformadores de predicados “strongest post-condition” e prove a correção do programa LPA.

```
[13]: from z3 import *

def strongest_postcondition_euclides():
    # Criar solver
    solver = Solver()
```

```

# Declarar variáveis simbólicas
a = Int('a')
b = Int('b')
r = Int('r')
r_linha = Int('r_linha')
s = Int('s')
s_linha = Int('s_linha')
t = Int('t')
t_linha = Int('t_linha')
q = Int('q')

# Estado inicial (pré-condição)
solver.add(a > 0)
solver.add(b > 0)

# Inicialização das variáveis
solver.add(r == a)
solver.add(r_linha == b)
solver.add(s == 1)
solver.add(s_linha == 0)
solver.add(t == 0)
solver.add(t_linha == 1)

# Invariante inicial
def adicionar_invariante(r, r_linha, s, s_linha, t, t_linha):
    return And(
        r == a * s + b * t,
        r_linha == a * s_linha + b * t_linha,
        Or(r_linha == 0, r_linha > 0)
    )

solver.add(adicionar_invariante(r, r_linha, s, s_linha, t, t_linha))

# Função para verificar SP após cada iteração
def verificar_sp_iteracao(r_old, r_linha_old, s_old, s_linha_old, t_old,
    ↪t_linha_old):
    r_new = r_linha_old
    r_linha_new = r_old % r_linha_old
    q_val = r_old / r_linha_old

    s_new = s_linha_old
    s_linha_new = s_old - q_val * s_linha_old

    t_new = t_linha_old
    t_linha_new = t_old - q_val * t_linha_old

    return (And(

```

```

        r_new == r_linha_old,
        r_linha_new == r_old - q_val * r_linha_old,
        s_new == s_linha_old,
        s_linha_new == s_old - q_val * s_linha_old,
        t_new == t_linha_old,
        t_linha_new == t_old - q_val * t_linha_old,
        adicionar_invariante(r_new, r_linha_new, s_new, s_linha_new, t_new,
→t_linha_new)
    ), (r_new, r_linha_new, s_new, s_linha_new, t_new, t_linha_new))

def verificar_pos_condicao(r_final, s_final, t_final):
    return And(
        r_final > 0,
        r_final == a * s_final + b * t_final,
        ForAll([Int('x'), Int('y')],
            Implies(
                And(Int('x') == a * Int('y') + b * Int('y'), Int('x') > 0),
                Int('x') >= r_final
            )
        )
    )

# Simular execução e verificar SP
def verificar_correcao(a_val, b_val):
    r_atual, r_linha_atual = a_val, b_val
    s_atual, s_linha_atual = 1, 0
    t_atual, t_linha_atual = 0, 1

    while r_linha_atual != 0:
        sp, (r_atual, r_linha_atual, s_atual, s_linha_atual, t_atual,
→t_linha_atual) = \
            verificar_sp_iteracao(r_atual, r_linha_atual, s_atual,
→s_linha_atual, t_atual, t_linha_atual)
        solver.add(sp)

    # Verificar pós-condição
    solver.add(verificar_pos_condicao(r_atual, s_atual, t_atual))

    return solver.check() == sat

return verificar_correcao

def testar_correcao():
    verificar = strongest_postcondition_euclides()

    casos_teste = [
        (48, 36),

```



```

        (17, 5),
        (21, 14)
    ]

    for a, b in casos_teste:
        print(f"\nTestando correcao para a={a}, b={b}")
        if verificar(a, b):
            print(f"Correção verificada com sucesso")
        else:
            print(f"Falha na verificação da correção")

if __name__ == "__main__":
    print("Verificando correção do algoritmo de Euclides estendido:")
    testar_correcao()

```

Verificando correção do algoritmo de Euclides estendido:

Testando correcao para a=48, b=36  
 Falha na verificação da correção

Testando correcao para a=17, b=5  
 Falha na verificação da correção

Testando correcao para a=21, b=14  
 Falha na verificação da correção